

Gaussian Process Regression for Esfahan Weather

A Technical Analysis of the Data Pipeline, Circular Wind-Direction Modeling, Evaluation
Design, and Reproducibility Architecture

Technical report derived exclusively from the supplied source implementation

Abstract

The project implements an end-to-end Gaussian Process Regression (GPR) pipeline for daily meteorological variables associated with Esfahan, Iran. The pipeline starts with hourly historical weather data from the Open-Meteo archive and produces a validated daily dataset, target-specific Gaussian-process models, chronological hold-out results, predictive uncertainty estimates, baseline comparisons, metadata, audit information, and packaged runtime artifacts. The central modeling choice is the periodic **ExpSineSquared** kernel with a fixed period of 365.25 days, which encodes annual seasonality. Separate scalar GPs are fitted for the individual targets rather than a joint multi-output process.

A key methodological detail is the treatment of wind direction as circular rather than linear data. The daily wind direction is computed through a vector mean in sine-cosine coordinates; two independent GPs then model the sine and cosine responses, and the predicted angle is reconstructed with `atan2`. An approximate angular uncertainty is obtained by a first-order delta-method calculation. Evaluation distinguishes the circular target from the remaining scalar variables and reports MAE, RMSE, normalized RMSE, R^2 , mean error, and empirical 95% interval coverage where appropriate.

This report describes what the code does, the rationale for its main computational choices, and the limitations that follow from its assumptions. No experimental outcomes are inferred beyond the information contained in the source. The report does not reproduce the project's figures or report performance values that are not present as executed outputs in the supplied file.

1. Introduction

The project treats the task as supervised time-series modeling, using a consecutive day index as the only temporal covariate to reconstruct and predict a collection of meteorological variables. The documented workflow downloads hourly weather observations, converts them to a validated daily dataset, fits independent Gaussian-process models, represents wind direction circularly, evaluates the models with a chronological 80/20 hold-out, and exports reproducible artifacts.

It is better viewed as a complete scientific-computing pipeline than as an isolated machine-learning function. Data acquisition, schema normalization, aggregation, validation, statistical modeling, uncertainty quantification, evaluation, artifact generation, and packaging are all part of the computational object being analyzed. This separation matters because a mathematically correct model is not enough if its input data are inconsistent or temporally misordered.

The file is a Colab-oriented generated source file. The beginning of the file contains a commented export of the intended standalone implementation, while later executable cells load an external `assignment_solution.py`, run its `run()` function, inspect the resulting files, and generate a clean notebook and submission package. The report treats the model implementation separately from the wrapper used to recover, execute, inspect, and package it.

2. Project Overview

Conceptually, the system follows this sequence of transformations:

1. define a geographic location, date interval, target variables, model settings, and reproducibility seed;
2. construct an Open-Meteo historical archive request for hourly variables in the **Asia/Tehran** time zone;
3. acquire the remote data as JSON or CSV and normalize its column schema;

4. optionally replace network acquisition with a deterministic synthetic fixture in test mode;
5. convert hourly observations to one daily row per calendar day;
6. compute the daily wind-direction mean on the circle rather than on the real line;
7. repair missing daily values by interpolation and forward/backward filling, then validate the resulting schema and temporal order;
8. split the dataset chronologically into training and testing segments;
9. fit one GPR model for each scalar target and two GPR models for wind direction, one for sine and one for cosine;
10. predict means and uncertainty on the hold-out interval and compute target-appropriate metrics;
11. fit final models on the complete daily dataset and evaluate them on a dense temporal grid for downstream artifact production;
12. compare against a seven-day persistence baseline;
13. write CSV, JSON, Markdown, PNG, notebook, script, and ZIP artifacts.

A central architectural feature is the separation between stages. Data acquisition does not directly perform modeling; aggregation produces a defined daily schema; validation is applied before modeling; evaluation is kept separate from final-model fitting; and packaging is performed only after required artifacts have been verified.

3. Technical Foundations

3.1 Gaussian process regression

A Gaussian process defines a probability distribution over functions. In regression, observations are represented as pairs (x_i, y_i) and the latent response is modeled by a stochastic function whose finite collections of values are jointly Gaussian. The code uses the scikit-learn `gaussian_process` module and `GaussianProcessRegressor`. Because the input consists only of the scalar `day_index`, the fitted process is a one-dimensional function of time for each target.

For a scalar target, the kernel used by `make_kernel()` is

$$k(x, x') = \sigma_f^2 \exp \left[-\frac{2 \sin^2(\pi|x - x'|/p)}{\ell^2} \right] + \sigma_n^2 \mathbf{1}[x = x'], \quad (1)$$

where ℓ is the characteristic length scale, p is the periodicity, and the white-noise term supplies a diagonal observation-noise component. In this implementation the multiplicative `ConstantKernel` is fixed at 1, the periodicity is fixed at $p = 365.25$ days, the initial length scale is 40 days, and the initial white-noise level is 0.1. The length scale and noise level are allowed to be optimized within the stated bounds.

The periodic kernel is useful when similarity should recur across an annual cycle. The key assumption is not that the weather series is perfectly periodic, but that observations separated by roughly one or more years should be more closely related than observations separated by unrelated offsets of the same size.

3.2 Posterior prediction

Let X denote the training inputs, y the corresponding training responses, K the kernel matrix over training inputs, and K_* the vector of kernel evaluations between training inputs and a test location x_* . With an additive white-noise term, the effective training covariance is

$$K_y = K + \sigma_n^2 I. \quad (2)$$

For a noise-free latent-function prediction, the posterior mean and variance take the standard form

$$\mu_* = K_*^\top K_y^{-1} y, \quad (3)$$

$$v_* = k(x_*, x_*) - K_*^\top K_y^{-1} K_*. \quad (4)$$

The code requests a predictive standard deviation from `predict(..., return_std=True)` and stores it as `posterior_std`. Because the kernel includes a white-noise component, this quantity reflects predictive uncertainty for the fitted stochastic model rather than uncertainty in the latent smooth function alone.

The final artifacts form Gaussian-style 95% intervals as

$$[\mu_* - 1.96 \sigma_*, \mu_* + 1.96 \sigma_*]. \quad (5)$$

The code measures empirical coverage of these intervals on the chronological test set. The factor 1.96 is a normal-approximation convention; the resulting intervals are not claimed to be exact frequentist confidence intervals.

3.3 Circular statistics for wind direction

Wind direction is an angle on the circle $[0, 360^\circ)$, not a linear scalar. A naive arithmetic mean can fail near the wrap-around boundary: values close to 359° and 1° should average to approximately 0° , not 180° . The code instead converts each angle ϕ to radians and computes

$$\bar{\theta} = \text{atan2}\left(\frac{1}{n} \sum_{i=1}^n \sin \phi_i, \frac{1}{n} \sum_{i=1}^n \cos \phi_i\right), \quad (6)$$

then converts the result back to degrees and maps it into $[0, 360)$.

For prediction, the daily direction is transformed to

$$s = \sin \theta, \quad c = \cos \theta, \quad (7)$$

and two independent scalar GPs are fitted to s and c . At prediction time the angle is reconstructed as

$$\hat{\theta} = \text{atan2}(\hat{s}, \hat{c}) \pmod{360^\circ}. \quad (8)$$

This preserves the topology of the circular variable at the representation level, even though the uncertainty calculation is performed approximately in the transformed coordinates.

3.4 Evaluation metrics

For ordinary scalar variables, the code reports mean absolute error, root mean squared error, normalized RMSE, coefficient of determination, and mean signed error. For observations y_i and

predictions $\hat{y}_i$,

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (9)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}, \quad (10)$$

$$\text{NRMSE} = 100 \frac{\text{RMSE}}{s_y}, \quad (11)$$

$$\text{MeanError} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i), \quad (12)$$

where s_y is the sample standard deviation of the true test values. The code uses the conventional R^2 implementation from scikit-learn for linear targets.

For wind direction, the prediction error is wrapped to the shortest signed angular displacement before its magnitude is taken:

$$e_i = |((y_i - \hat{y}_i + 180^\circ) \bmod 360^\circ) - 180^\circ|. \quad (13)$$

The resulting MAE and RMSE measure angular discrepancy rather than linear distance on the real line. R^2 is intentionally left undefined for this target.

4. Data and Input Processing

4.1 Configuration

The configuration block establishes the reproducibility and experimental scope. The seed is 1. The geographic coordinates are latitude 32.6546 and longitude 51.6680, and the requested date interval is 2022-05-01 through 2023-01-19. The test fraction is 0.20, the nominal number of optimizer restarts is 10, the dense prediction grid contains 11,000 points, and network requests use a 60-second timeout.

The hourly input schema contains meteorological variables such as temperature, humidity, dew point, apparent temperature, precipitation, rain, snowfall, wind speed, wind gusts, wind direction, shortwave radiation, cloud cover, pressure, evapotranspiration, vapor-pressure deficit, and shallow soil moisture. The daily target schema contains 23 derived variables, including extrema, means, sums, duration counts, and the circular mean of wind direction.

4.2 Remote acquisition and schema normalization

`build_hourly_url()` constructs an Open-Meteo archive URL using URL encoding. The request explicitly sets the time zone to `Asia/Tehran` and selects Celsius, km/h, and millimetres for the relevant units. This reduces the risk of a unit mismatch between the data provider and downstream processing.

The parser is intentionally tolerant of response formatting. `download_raw_csv()` first examines the raw response text. If it appears to be JSON, the code checks for an API error and then constructs a DataFrame from the `hourly` payload. Otherwise it searches for a line containing the field `time` and treats that line as the CSV header. `normalize_column_name()` strips whitespace and surrounding quote characters and removes parenthetical annotations from names. The final data-loading stage checks both non-emptiness and the presence of the time column before

writing the raw CSV artifact.

4.3 Offline validation fixture

A deterministic fixture is available when `ASSIGNMENT_TEST_MODE` is set to 1.

It creates 270 days of hourly data beginning at the same start date and uses a fixed NumPy generator seeded with 1. The fixture contains structured seasonal components plus Gaussian perturbations, with clipping or non-negativity constraints where the meteorological quantity requires them.

This path is intended for software validation, not scientific inference. It decouples syntax, schema, aggregation, model-shape, and packaging tests from network availability. The normal pipeline sets test mode to 0, so end-to-end execution uses the online historical data.

5. Hourly-to-Daily Transformation

5.1 Time parsing and ordering

The raw time field is converted with `pandas.to_datetime(..., errors='coerce')`. Rows with invalid timestamps are removed, the data are sorted chronologically, and the temporary daily key is defined by flooring timestamps to the day boundary. Every meteorological hourly field is coerced to numeric form with invalid entries represented as missing values.

5.2 Daily aggregation rules

The aggregation logic is variable-specific rather than generic. Temperature and apparent temperature use daily maxima, minima, and means. Dew point, relative humidity, cloud cover, pressure, and shallow soil moisture use daily means. Precipitation, rain, snowfall, and FAO reference evapotranspiration use sums. Maximum wind speed and maximum gust use daily maxima.

Two duration quantities are derived by counting hourly observations that exceed explicit thresholds. `precipitation_hours` counts hours for which precipitation is strictly positive. `sunshine_duration_hours` counts hours for which shortwave radiation exceeds 120. These are operational definitions based on thresholds rather than direct measurements of the corresponding physical durations.

Daily shortwave radiation is converted from the hourly rate-like quantity by summing hourly values, multiplying by 3600 seconds per hour, and dividing by 10^6 to express the result in megajoules per square metre:

$$E_{\text{day}} = \frac{3600}{10^6} \sum_{h=1}^H R_h. \quad (14)$$

This conversion is implemented directly in the aggregation lambda.

5.3 Missing-value treatment

After merging the circular wind-direction result into the daily table, all target columns are coerced to numeric values and passed through linear interpolation with two-sided limit filling, followed by forward fill and backward fill. This combination ensures that isolated gaps and edge gaps are replaced before model fitting.

Interpolation uses information from both sides of a missing interval, so it is a preprocessing step over the observed dataset rather than a causal forecasting procedure. Because the split

is performed after this imputation step, the source does not enforce a train-only imputation boundary. Here, imputation is treated as part of dataset reconstruction. A strict forecasting study may instead impose a no-future-information rule for missing-value repair.

6. Dataset Validation and Temporal Design

6.1 Schema and integrity checks

The `validate_dataset()` function requires at least 200 daily rows. It verifies that the column order exactly matches the expected daily schema; dates are monotonically increasing and unique; `day_index` begins at zero and advances by exactly one; all target values are finite; and the wind direction remains in the half-open interval $[0, 360)$.

This validation layer goes beyond a simple shape check by verifying invariants that matter to the statistical model. In particular, the GP receives `day_index` as a numerical coordinate, so non-monotone or duplicated dates would have changed the intended meaning of the input space.

6.2 Chronological hold-out

The split function computes

$$n_{\text{train}} = \lfloor n(1 - 0.20) \rfloor, \quad (15)$$

with additional guards that keep at least two observations on each side. Training observations are the initial prefix and test observations are the subsequent suffix. No random shuffling is used.

This is a suitable design for temporal reconstruction or forecasting-style evaluation because test observations are not interleaved with training observations. It does not, however, establish that every preprocessing step is causal with respect to the split; the earlier interpolation stage occurs before the split. The distinction between temporal splitting and fully leakage-free preprocessing is important.

7. System Architecture and Code Organization

The code is organized into explicit functional layers. Table 1 summarizes the principal public functions and the role each plays in the end-to-end pipeline.

Table 1: Principal functions and responsibilities.

Function	Responsibility
<code>seed_everything</code>	Seeds Python’s <code>random</code> module and NumPy.
<code>make_dirs</code>	Creates the data, tables, figures, and logs directory structure.
<code>build_hourly_url</code>	Encodes the Open-Meteo hourly archive request.
<code>normalize_column_name</code>	Canonicalizes incoming field names.
<code>parse_remote_csv</code>	Locates a CSV header and parses the remote text.
<code>download_raw_csv</code>	Switches between real acquisition and the deterministic fixture.
<code>circular_mean_degrees</code>	Computes a daily circular mean for wind direction.
<code>aggregate_hourly_to_daily</code>	Builds the validated daily target table.
<code>validate_dataset</code>	Enforces schema, ordering, finiteness, and direction invariants.
<code>chronological_split</code>	Produces ordered training and test arrays.
<code>make_kernel</code>	Defines the fixed-period periodic GP covariance plus white noise.
<code>fit_scalar_gp</code>	Instantiates and fits a scalar GPR model.
<code>fit_direction_models</code>	Fits separate GPs to sine and cosine wind-direction responses.
<code>predict_direction</code>	Reconstructs angles and approximates angular uncertainty.

Function	Responsibility
<code>scalar_metrics</code>	Computes the standard scalar regression metrics.
<code>build_metrics</code>	Applies target-specific evaluation logic and interval coverage.
<code>fit_and_evaluate</code>	Performs the chronological test evaluation and writes predictions.
<code>fit_final_models</code>	Retrains on the complete dataset and creates dense predictions.
<code>weekly_baseline</code>	Evaluates a seven-day persistence baseline.
<code>write_metadata</code>	Records environment, dataset, target, and run settings.
<code>write_audit</code>	Writes the methodological and validation audit report.
<code>create_package</code>	Collects artifacts into a ZIP archive.
<code>run</code>	Orchestrates the full computational workflow.

This separation of responsibilities is useful both scientifically and from a software-engineering perspective. Each transformation can be inspected independently, and the main `run()` function acts as a readable control plane rather than containing all data manipulation inline.

8. Detailed Methodology

8.1 Scalar Gaussian-process fitting

For every scalar target, `fit_scalar_gp()` constructs a new `GaussianProcessRegressor` with the common kernel. The regressor uses `normalize_y=True`, a small numerical diagonal term `alpha=1e-8`, and a fixed random state of 1. Hyperparameter optimization is enabled through `n_restarts_optimizer`; the helper `effective_restarts()` returns 0 in test mode and the configured value 10 otherwise.

The common kernel gives all targets the same structural prior on temporal periodicity while allowing each model to learn its own length scale and noise level. This balances target-specific fitting with a shared kernel design.

8.2 Wind-direction modeling

The target `wind_direction_mean` is routed through a separate branch in both training and prediction. The target angle is converted to radians, and its sine and cosine are fitted independently. At test time, the two predictive means are passed to `atan2`.

The uncertainty calculation defines

$$r^2 = \hat{s}^2 + \hat{c}^2, \quad (16)$$

and uses the approximation

$$\text{Var}(\hat{\theta}) \approx \frac{\hat{c}^2 \sigma_s^2 + \hat{s}^2 \sigma_c^2}{r^4}, \quad (17)$$

followed by conversion from radians to degrees. This is a first-order delta-method approximation for the angle obtained from a pair of noisy Cartesian coordinates. The implementation clips r^2 below by 10^{-12} to avoid division by values that are numerically too small.

This approximation is consistent with the model structure, but it is not a full circular posterior. The two GP predictions are treated through their marginal standard deviations, and the code does not model a covariance term between the sine and cosine prediction errors. The resulting angular standard deviation is therefore an approximation derived from the two scalar predictive uncertainties.

8.3 Test-time uncertainty and coverage

For scalar targets, the code forms lower and upper bounds as $\hat{y} \pm 1.96\hat{\sigma}$ and reports the proportion of test observations contained in those intervals. For wind direction, it compares the circular absolute error with 1.96 times the angular standard deviation.

Coverage is empirical and descriptive rather than a theoretical guarantee. A value near 95% would be compatible with well-calibrated Gaussian-style intervals, but the source does not establish theoretical

calibration.

9. Optimization and Learning Procedure

The GPR implementation delegates hyperparameter optimization to scikit-learn. The kernel’s periodicity is fixed, as specified by `periodicity_bounds='fixed'`, and the constant-kernel amplitude is fixed. The principal learned kernel parameters are therefore the length scale and white-noise level. The bounds are broad: length scale $\ell \in [0.1, 100000]$ and noise level $\sigma_n^2 \in [10^{-5}, 10]$ at the kernel-parameter level used by the estimator.

The function `effective_restarts()` suppresses optimizer restarts in test mode. This keeps deterministic fixture validation focused on the computational path without incurring the full hyperparameter-search cost. Normal execution restores the configured ten restarts.

There is no explicit early-stopping rule, manual gradient-update loop, or custom optimizer. Hyperparameter optimization therefore follows the estimator’s internal marginal-likelihood procedure. The source does not expose the final optimized hyperparameter values, so they are not inferred here.

10. Final-Model Fitting and Dense Evaluation Grid

After chronological evaluation, `fit_final_models()` refits every target-specific model on the complete daily dataset. This stage is separate from the held-out evaluation models: it gives up test-set separation in exchange for a model trained on all available observations for downstream use.

The dense grid is generated as

$$x_j = \text{linspace}(x_{\min}, x_{\max}, 11000)_j, \quad (18)$$

and each target receives predicted mean, predictive standard deviation, and Gaussian-style lower and upper 95% bounds. These quantities are written to `dense_predictions.csv`. The code also saves `model_input_daily.csv`, which preserves the daily date and target values used in the final fit.

The dense grid is used for smooth presentation or downstream analysis; it does not add observed information. It should not be interpreted as 11,000 new observations.

11. Baseline and Comparative Evaluation

The project includes a seven-day persistence baseline. For each test day i , the baseline obtains the target vector from the observation seven rows earlier in the chronologically ordered series. When the required lag would reach before the beginning of the table, it falls back to the final training observation.

The comparison quantity is the percentage reduction in RMSE relative to baseline:

$$\text{Improvement}(\%) = 100 \frac{\text{RMSE}_{\text{base}} - \text{RMSE}_{\text{GP}}}{\text{RMSE}_{\text{base}}}, \quad (19)$$

provided that the baseline RMSE is positive.

One subtle property of this baseline is that, after the first seven test observations, its inputs come from earlier observations within the test segment itself. Thus it is a seven-day persistence benchmark with access to the realized history available by each prediction time, not a single static value carried across the whole test horizon.

12. Implementation Details

12.1 Reproducibility

The pipeline records the Python version, operating-system platform, NumPy, pandas, and scikit-learn versions, together with seed, coordinates, date range, target list, data source, and test-mode status. These

values are written to `runtime_metadata.json`. The pipeline also writes `run_summary.json`, an audit Markdown file, and a ZIP package containing the selected scripts, notebook material, data, and outputs.

Reproducibility is based on explicit configuration and metadata rather than on a formal environment lockfile. The source does not include a `requirements.txt`, package lock, container definition, or exact hardware specification.

12.2 Error handling and validation

Network requests use a timeout and call `raise_for_status()`. API responses are checked for an explicit error field and for the presence of hourly data. The aggregation stage verifies that every expected hourly column exists. The dataset validator checks both structure and numeric integrity. The execution wrapper also verifies that the major output files exist before reporting successful completion.

This layered validation is a strength of the implementation because failures are surfaced near the boundary where they occur. For example, a malformed remote CSV is rejected before it can silently propagate into model fitting.

12.3 Colab and script interoperability

The file is designed to work in Google Colab while still producing a standalone `assignment_solution.py`. The wrapper attempts to recover a `%writefile assignment_solution.py` cell from notebook history when the external script is missing. It then uses `runpy.run_path()` with a non-`__main__` name so that the script's definitions can be imported without executing its standalone block.

The export stage constructs a fresh notebook with an implementation cell, a run cell, and an export cell. This packaging-oriented design turns an interactive development artifact into a reproducible submission unit rather than a minimal notebook.

13. Execution Workflow

The top-level `run()` function provides the clearest view of the program's control flow. It first starts a wall-clock timer and seeds the random generators. It then resolves the output root and creates the directory tree. The raw weather file is downloaded, converted to the daily dataset, and written to disk. Metadata are written immediately after preprocessing.

Next, `fit_and_evaluate()` performs the chronological split, trains the target-specific GPs, predicts the test segment, and writes the first test-metrics and test-predictions tables. The final models are then trained on all daily observations, the seven-day baseline is evaluated, and the baseline results are merged into the main metrics table. It computes a relative RMSE improvement field and rewrites the final metrics CSV.

The pipeline then computes a dataset summary containing mean, standard deviation, minimum, and maximum for every target, writes the dense final-model predictions, generates figures, writes the audit report, and produces a runtime summary JSON file. The standalone branch can subsequently package the script, notebook, audit report, data, and outputs into a ZIP archive.

The notebook-level wrapper adds another verification layer: after `run()` returns, it checks for the daily CSV, test metrics, and dense predictions; prints the metrics table; inspects the daily dataset schema; and displays the generated figures. The supplied report deliberately does not reproduce those figures, in accordance with the requested publication format.

14. Design Decisions and Rationale

14.1 Periodic kernel with fixed annual period

The central modeling choice is the annual periodic kernel. Fixing the period at 365.25 days encodes an explicit seasonal hypothesis while leaving the local smoothness and noise characteristics to be optimized.

This is reasonable for meteorological time series because many weather variables exhibit annual recurrence, even though their trajectories are not perfectly repetitive.

The data interval covers less than one complete annual cycle, so the annual periodic prior is partly imposed by modeling choice rather than identified from multiple observed cycles. This is not an implementation error, but it limits how strongly annual recurrence can be assessed.

14.2 Independent target models

The code fits one GP per scalar variable and a pair of GPs for wind direction. This keeps the implementation simple and makes uncertainty computation target-specific. The trade-off is that correlations among meteorological variables are not represented. For example, the model does not jointly learn that temperature, dew point, humidity, and apparent temperature are physically related.

14.3 Circular representation

Using sine and cosine responses is a clear methodological improvement over directly regressing angles with a Euclidean loss. The wrap-around issue is handled at both the aggregation and evaluation stages, not only at prediction time. This demonstrates awareness that data representation, learning procedure, and metric definition must agree for a circular variable.

14.4 Chronological rather than random evaluation

The test split is the final suffix of the time series. This is appropriate for temporal dependence because random shuffling can leak future seasonal structure into the training set. The design is not equivalent to rolling-origin evaluation or a multi-year out-of-sample study, but it preserves temporal ordering more faithfully than a random split.

15. Computational Considerations

For a standard exact Gaussian process with n training observations, the dominant linear-algebra operations scale cubically in time and quadratically in memory. In asymptotic notation,

$$T(n) = O(n^3), \quad M(n) = O(n^2). \quad (20)$$

The stated date interval contains only a few hundred daily observations, so exact GPR remains computationally plausible. However, the total modeling cost is multiplied by the number of target-specific models, and by additional optimizer restarts. The project therefore prioritizes methodological simplicity and exact GP inference over large-scale scalability.

The dense grid of 11,000 prediction points does not change the cubic training complexity, but it increases the cost of posterior evaluation and artifact storage. The use of a one-dimensional input is computationally modest, yet it also means the model cannot exploit explanatory covariates beyond elapsed time.

The offline fixture and zero-restart test mode are specifically useful for software-validation speed. In normal execution, ten optimizer restarts can materially increase fitting time, especially because the process is repeated independently across 23 targets and two transformed wind-direction responses.

16. Reproducibility and Reliability

The code contains several positive reproducibility mechanisms. The random seed is explicit, model random state is fixed, the dataset's date range and geographic coordinates are configured in one place, and runtime versions are persisted. The raw and daily CSVs are preserved, and the final package records both code and audit material.

The strongest reliability feature is the layered contract between stages. Acquisition creates a raw artifact, aggregation creates a validated daily artifact, model fitting consumes the validated representation,

evaluation writes explicit metrics and predictions, and packaging occurs only after files are checked. This design reduces the chance that a partially completed run will be mistaken for a valid analysis.

The remaining limitation is environment reproducibility. Recording package versions is informative, but it does not itself guarantee binary or dependency-level reproducibility. Network data are also an external dependency, so repeating the same code later may retrieve an updated archival representation or otherwise depend on provider-side behavior not controlled by the repository.

17. Limitations and Technical Considerations

17.1 Incomplete annual coverage

The configured period from 2022-05-01 to 2023-01-19 spans substantially less than a full 365.25-day cycle. The annual periodicity is therefore a strong prior rather than a pattern learned across repeated annual cycles. A longer multi-year dataset would provide a stronger empirical basis for estimating and validating seasonal recurrence.

17.2 Data provenance

The source identifies the Open-Meteo archive as the provider of historical reanalysis data. The code treats these data as the modeling source and does not include an additional measurement-quality correction or comparison against a local station. The data are therefore not characterized as direct station observations.

17.3 Imputation relative to the split

Interpolation and edge filling are performed before the chronological train/test split. In a strict forecasting study where missing future values must be handled using only past information, this ordering can permit information from later timestamps to influence earlier missing-value reconstruction. The source does not document a causal imputation policy, so this is best treated as a methodological consideration.

17.4 Approximate angular uncertainty

The wind-direction uncertainty calculation is based on a first-order delta method. It does not construct a circular predictive distribution and does not include an explicit covariance between the sine and cosine predictive errors. The approximation can become unstable when the predicted Cartesian vector has very small magnitude; the code therefore imposes a numerical floor on r^2 .

17.5 Independent rather than multi-output modeling

The models do not exploit cross-target covariance. This is a deliberate simplification, but it prevents the model from using cross-target information such as the correlation between temperature and humidity or between wind speed and gusts.

17.6 Evaluation scope

The source computes a single chronological 80/20 hold-out, a seven-day baseline, and interval coverage. It does not perform rolling-origin cross-validation, repeated temporal splits, formal uncertainty calibration diagnostics, or statistical significance testing. The report does not claim generalization beyond the evaluated hold-out design.

17.7 Figure generation and report scope

The implementation contains a `save_figures()` function that produces target trajectories and metric bar charts. Those artifacts are part of the software pipeline but are intentionally omitted from this report. The report documents the program's capabilities without embedding every runtime artifact.

18. Overall Technical Assessment

The code is technically coherent as an end-to-end scientific-computing pipeline. Its main strengths are the separation of data preparation and modeling, chronological evaluation, explicit handling of circular wind direction, predictive uncertainty, and validation and packaging infrastructure. It also provides good auditability by recording runtime environment metadata, writing a methodological audit, and supporting an offline deterministic fixture.

The modeling formulation is deliberately simple: every target is a one-dimensional function of time under the same annual-periodic kernel family. This makes the behavior interpretable and computationally manageable, but it also limits the scientific expressiveness of the model. There is no multivariate covariate structure, no explicit cross-target dependency, and no repeated-year validation design. These are scope limitations rather than evidence of poor implementation.

From a software-engineering perspective, the functional decomposition is strong: the main orchestration function is readable, input contracts are checked, external failures are surfaced, and artifacts are written in predictable locations. From a research-methodology perspective, the clearest areas for refinement are train-only preprocessing, multi-period temporal evaluation, and a richer probabilistic treatment of circular uncertainty.

19. Conclusion

The project implements a complete weather time-series modeling workflow built around exact Gaussian Process Regression with an annual periodic kernel. Its data pipeline converts hourly Open-Meteo observations into a validated daily representation, derives meteorologically meaningful aggregates, and explicitly preserves the circular geometry of wind direction. The learning stage fits independent Gaussian processes to scalar targets and to the sine/cosine embedding of the directional target. Evaluation is chronological, uncertainty is reported through predictive standard deviations and Gaussian-style 95% intervals, and a seven-day persistence baseline provides a simple reference point.

The project also demonstrates attention to reproducibility and software reliability. Configuration is centralized, random seeds are explicit, schema and numerical invariants are validated, runtime metadata are recorded, output artifacts are checked, and the final materials are packaged for portable execution. At the same time, the code makes several assumptions that should remain visible to a research reader: the available observation period is shorter than a full annual cycle, preprocessing imputation precedes the temporal split, wind-direction uncertainty is approximate, and independent target models ignore cross-variable dependencies.

Taken as a whole, the code is best characterized as a well-structured, research-oriented implementation of a periodic Gaussian-process methodology rather than as a production-scale forecasting system. Its value for an academic portfolio lies in the combination of mathematical modeling, careful data handling, uncertainty-aware evaluation, and reproducible computational practice.